

CSC490: Assignment 2

Team Members

Name	Student ID
Aarya Prakash	1008790257
Derek Huynh	1008849052
Merrick Liu	1008951920
Rhys Balevicius	1000801974

University of Toronto
Department of Computer Science

1 Aspirational Datasets

Our goal is to improve the efficacy of AI coding agents, which may produce incorrect changes because they lack relevant context that exists outside their immediate working scope. To build and evaluate systems that surface this missing context, the ideal dataset would consist of tuples pairing a coding task with a point-in-time snapshot of the full software environment: the target repository, its transitive dependencies, documentation, and version history along with a gold label identifying the precise, minimal set of non-local information an agent would have needed to produce a correct change. This label is the core artifact: it answers the question *"what did the agent need to know, but didn't, in order to get this right?"* and it spans multiple axes of context, including interface contracts and type signatures across module and library boundaries, downstream callers and invariants that a local change might violate, architectural conventions and design rationale recoverable from project history, and relevant documentation or specifications from external dependencies.

Each entry would additionally include a failed agent attempt (produced without sufficient context), the corresponding corrected code (produced by a human or by an agent given adequate context), and the failure signal linking the two such as a test failure, type error, or behavioural regression. The causal structure of this data is what makes it powerful: the delta between the failed and successful attempts is attributable to the presence or absence of the labelled context. With such a dataset, one could train and evaluate any retrieval or reasoning approach against a ground-truth definition of what "the right context" is for a given task, and measure whether surfacing it actually changes agent outcomes.

Aspirational Dataset 1: Context-Attributed Agent Failures

Each row captures a coding task where an agent failed, the corrected outcome, and a precise attribution of what non-local context was missing.

Task		
Field	Type	Notes
repo	<code>str</code>	Target repository
repo_snapshot	<code>commit_sha</code>	Frozen state of repo at task time
dependency_snapshot	<code>[package@version]</code>	Pinned versions of all transitive dependencies
task	<code>str</code>	Natural language description of the coding task
agent_attempt	<code>AgentAttempt</code>	Incorrect attempt record
failure_signal	<code>FailureSignal</code>	Failure manifestation
corrected_diff	<code>Diff</code>	Correct change
missing_context	<code>[MissingContext]</code>	Required unseen context

Aspirational Dataset 2: Interface Contracts and Cross-Boundary Specifications

Each row describes a function or module boundary, its (possibly implicit) contract, and the set of callers/consumers that depend on that contract across repo and library boundaries.

Function		
Field	Type	Notes
repo	str	Repository
repo_snapshot	commit_sha	Frozen state
function	fully_qualified_name	Canonical name
defined_in	{file, span}	Definition location
contract	Contract	Behavioural specification
consumers	[Consumer]	Call sites
known_violations	[KnownViolation]	Historical breaks

Aspirational Dataset 3: Downstream Impact Chains

Each row captures a local change and the full causal chain through which it produces a failure at a distant location.

ChangeImpact		
Field	Type	Notes
repo	str	Repository
repo_snapshot	commit_sha	Frozen state
local_change	Diff	Triggering modification
change_location	{file, function}	Origin of change
failure_location	{file, function, test}	Where breakage surfaced
causal_chain	[CausalNode]	Propagation path
chain_length	int	Hop count
minimal_context	[{file, span}]	Minimal foresight set

Aspirational Dataset 4: Architectural Conventions and Project Norms

Each row encodes an implicit or explicit convention in a codebase, examples of code that follow it, and examples of agent-generated code that violates it.

ConventionSchema

Field	Type	Notes
repo	str	Repository
repo_snapshot	commit_sha	Frozen state
convention	Convention	Normative pattern
conforming_examples	[{file, span}]	Correct usages
violations	[Violation]	Broken instances

Aspirational Dataset 5: Design Rationale and Temporal Context

Each row links a region of code to the historical decisions that explain *why* it is the way it is: the context an agent needs to avoid "cleaning up" intentional complexity.

CodeHistory

Field	Type	Notes
repo	str	Repository
repo_snapshot	commit_sha	Frozen state
code_region	{file, span}	Location
current_form	str	Present code
history	[HistoryEntry]	Prior commits
rationale	Rationale	Design justification
naive_refactors	[NaiveRefactor]	Tempting but incorrect changes

Aspirational Dataset 6: Cross-Repository Dependency Context

Each row captures a usage of an external library and the external documentation, source, or changelog context needed to use it correctly.

DependencyUsage

Field	Type	Notes
consumer_repo	str	Consuming repo
repo_snapshot	commit_sha	Frozen state
dependency	{name, version}	External package
usage_location	{file, span}	Call site
api_used	fully_qualified_name	Referenced symbol
required_context	[RequiredContext]	External knowledge
version_sensitivity	VersionSensitivity	Version constraints
misuse_examples	[MisuseExample]	Incorrect usages

2 Reality Check

The ideal schema above asks for five things per task: a task statement, a point-in-time code/dependency snapshot, a failure signal, a corrected change, and explicit attribution of the missing non-local context. No single public source gives all five, so the practical path is to combine (1) existing benchmarks, (2) mined web data, and (3) synthetic generation pipelines.

Dataset	Description	Commentary
SWE-bench	2,294 task instances from 12 Python repos. Each instance includes issue, PR, and test patch.	Primary evaluation benchmark. Run agents with/without MCP service and measure resolution-rate delta.
SWE-bench-Live	Continuously updated variant of SWE-bench with more recent issues post-training-cutoff. Same schema (issue, PR, test patch).	Anti-contamination eval set. Prevents inflated scores from benchmark leakage into foundation model training data. Use as a held-out evaluation set once our system is tuned on original SWE-bench.
SWE-smith	Toolkit for generating synthetic SWE-bench-style instances via automated code transformations (bug injection, context removal). Can produce instances at scale on arbitrary repos.	Scale up evaluation + training data. Original SWE-bench is only 2K instances; SWE-smith enables orders-of-magnitude more. Useful for stress-testing retrieval across diverse repo structures and for generating Dataset 1 (context-attributed failures) synthetically with known ground-truth context.
SWE-Gym-Raw	Execution environments for SWE-bench instances. Provides Docker setups with pinned dependencies for running tests against agent patches.	Evaluation infrastructure (not a dataset). Enables reproducible eval harness execution and avoids building custom sandboxes. Low priority for data ingestion; high priority for evaluation.
SWE-rebench	Re-evaluated SWE-bench leaderboard with stricter pass/fail criteria (avoids false positives from flaky tests, overcounting, etc.).	Meta-resource for evaluation calibration. Not a dataset to ingest. Use their methodology to ensure our eval harness produces trustworthy results. Reference only.
RepoBench	Benchmark for repo-level code completion. Tasks require cross-file context (imports, related functions in other files) to complete a function body correctly. ~11K instances across Python and Java.	Directly relevant for code graph retrieval evaluation. Tests whether the system surfaces correct cross-file context (e.g., <code>get_callers</code> , <code>get_contract</code>). Strong eval set for the indexing layer independent of the full MCP pipeline. Covers Dataset 2 (contracts/cross-boundary specs).
The Stack v2	67.5 TB of permissively licensed source code from Software Heritage, spanning hundreds of languages.	Training resource for embeddings (not an eval dataset). Suitable for training or fine-tuning code embeddings (e.g., Milvus semantic search). Also useful for large-scale convention mining (Dataset 4). Heavy to process; use targeted subsets.
Software Heritage Graph	Complete graph of publicly known software development: repos, commits, directories, files, and their relationships. Available as node+edge export.	Dependency and impact analysis at scale. Can support Dataset 3 (downstream impact chains) and Dataset 6 (cross-repo dependency context) via ecosystem-level call/import graphs. Very heavy; best used via targeted API or BigQuery queries rather than full ingestion.

Dataset	Description	Commentary
GH Archive	Complete GitHub event stream since 2011 (PushEvents, PullRequestEvents, IssuesEvents, IssueCommentEvents, etc.), available as JSON via BigQuery or direct download.	Primary scraped training source. Mine issue→PR→fix chains at scale. Review comments often contain explicit missing-context signals (e.g., “this breaks because of X in module Y”). Feeds Datasets 1 and 5 (context-attributed failures, design rationale) and supports mental model construction via repo evolution. High priority for the offline pipeline.
deps.dev	Google’s dataset of dependency relationships across package ecosystems (npm, PyPI, Go, Maven, Cargo, etc.), available via BigQuery. Includes dependency graphs, versions, advisories, and licenses.	Feeds <code>get_dependency_context</code> and Dataset 6. Provides version graphs, breaking-change signals, and advisory data when indexing repo dependencies. High priority for the dependency index.
CodeSearchNet	code+docstring pairs across six languages (Python, Java, JS, PHP, Go, Ruby), originally released for training code search models.	Training data for embedding models. Provides high-quality code+NL pairs for contrastive learning (e.g., Milvus semantic search). Also useful for evaluating NL→code retrieval performance (e.g., <code>get_context_for_change</code>). Medium priority.
Mutation-induced failing patches	Generate via <code>mutmut</code> (or similar) on repos with tests. Inject mutations, run tests, retain failing mutants. Produces (<code>mutation.diff</code> , <code>failing_tests</code> , <code>original.code</code>) triples.	High-volume source for Datasets 1 and 3. Controlled perturbations provide exact change provenance and traceable failure paths. Suitable for large-scale retrieval-ranker training (“what context would have prevented this mistake?”). Generation should run on repos already in the data lake.
Mutation-repair pairs with context budgets	For each mutated bug, collect an agent fix under limited context and under augmented context; compare outcomes.	Highest-value synthetic dataset. Directly captures the causal signal: did context X change the outcome from wrong to right? Provides ground truth for training the ranking in <code>get_context_for_change</code> . Expensive to generate (requires agent runs), but highest signal-to-noise ratio.
Documentation generation	Generate or extract docstrings and use them to train similarity between code semantics and natural language descriptions.	Secondary priority. Supports embedding fine-tuning to better align code and documentation (e.g., Milvus). Improves <code>get_dependency_context</code> and other NL-based retrieval. Can be deferred until semantic search becomes the bottleneck.
Closed issues + closing PRs + merge diffs	Mine via the GitHub GraphQL API: issue text, discussion threads, linked PRs, review comments, and commit diffs. Produces (<code>task.description</code> , <code>discussion_context</code> , <code>fix_diff</code>) tuples.	Primary scraped dataset (high priority). Closest approximation to naturally occurring Dataset 1 instances. Review comments often contain explicit missing-context annotations (“this breaks X because Y”). Feeds retrieval ranking, mental model construction, and convention mining. Should be prioritized in the offline pipeline.

- 3 Data-processing pipelines
- 4 IaC Implementation
- 5 Disaster Recovery Demonstration